

Universidad del Valle de Guatemala

Facultad de Ingeniería
Modelos Cuantitativos de Información

Análisis Predictivo de Popularidad en Repositorios de Ciencia de Datos en GitHub

Proyecto Final

Integrantes

Carlos Aldana

Carlos Angel

Diego Monroy

Guatemala, mayo de 2026

1. Introducción

El ecosistema open source ha crecido de forma considerable durante la última década, especialmente en el área de Ciencia de Datos e Inteligencia Artificial. Plataformas como GitHub concentran miles de proyectos que van desde pequeñas librerías experimentales hasta frameworks de uso masivo como TensorFlow o PyTorch. En este contexto surge la pregunta de qué factores determinan que un repositorio logre una adopción significativa dentro de la comunidad.

El presente proyecto tiene como objetivo aplicar técnicas de análisis exploratorio y modelado predictivo para identificar los factores asociados a la popularidad de repositorios de Ciencia de Datos en GitHub. La popularidad se mide a través del número de estrellas (stars), que es el mecanismo que los usuarios utilizan para marcar proyectos de interés dentro de la plataforma.

Para ello se construyó un dataset directamente desde la GitHub REST API, se realizó un análisis exploratorio exhaustivo y se entrenaron distintos modelos de machine learning, seleccionando el de mejor desempeño y optimizando sus hiperparametros con búsqueda bayesiana mediante Optuna.

2. Contexto y Origen de los Datos

Los datos fueron obtenidos directamente mediante la GitHub REST API utilizando un script en Python que realizo búsquedas sobre repositorios públicos relacionados con los siguientes temas:

- Data Science
- Machine Learning
- Artificial Intelligence
- Deep Learning
- NLP (Procesamiento de Lenguaje Natural)
- Computer Vision

La recopilación se llevó a cabo de forma automática a través del script `scripts/fetch_github_data.py`, que iteraba sobre los resultados paginados de la API y almacenaba la información en formato CSV. La elección de GitHub como fuente de datos responde a que es la plataforma de desarrollo colaborativo más utilizada a nivel mundial, lo que garantiza que los datos reflejan dinámicas reales del ecosistema open source.

Este conjunto de datos resulta de interés para el equipo dado que los tres integrantes trabajamos con herramientas de Ciencia de Datos de manera regular, y entender que proyectos logran mayor visibilidad es relevante tanto desde una perspectiva académica como profesional.

3. Descripción del Dataset y sus Variables

El dataset final cuenta con 1,573 repositorios y 25 variables. Después de eliminar valores atípicos extremos (por encima del percentil 99 en la variable objetivo), se trabajó con 1,557 registros para el proceso de modelado.

A continuación se describen las variables principales utilizadas en el análisis:

Variable	Tipo	descripción
stars	Numérica (objetivo)	Numero de estrellas del repositorio

forks	Numérica	Veces que fue bifurcado el repositorio
watchers	Numérica	Usuarios que siguen el repositorio
open_issues	Numérica	Issues abiertos actualmente
size	Numérica	Tamaño del repositorio en KB
language	Categórica	Lenguaje de programación principal
topics_count	Numérica	Numero de tópicos asignados
repo_age_days	Numérica	Antigüedad del repositorio en días
days_since_update	Numérica	Días desde la última actualización
stars_per_day	Derivada	Tasa de crecimiento diaria de estrellas
forks_per_day	Derivada	Tasa de forks por día de vida
has_wiki / has_issues	Booleana	Indica si tiene wiki o issues habilitados
description_length	Numérica	Longitud de la descripción del repositorio

Tabla 1. Variables principales del dataset.

4. Análisis Exploratorio de Datos

4.1 Distribución de la Variable Objetivo

La variable stars presenta una distribución altamente asimétrica positiva, lo cual es típico en plataformas open source donde pocos proyectos concentran la mayor parte de la atención. Al aplicar la transformación logarítmica $\log(\text{stars} + 1)$ se observa una distribución mucho más cercana a la normal, lo que justifica su uso para mejorar el ajuste en modelos lineales.

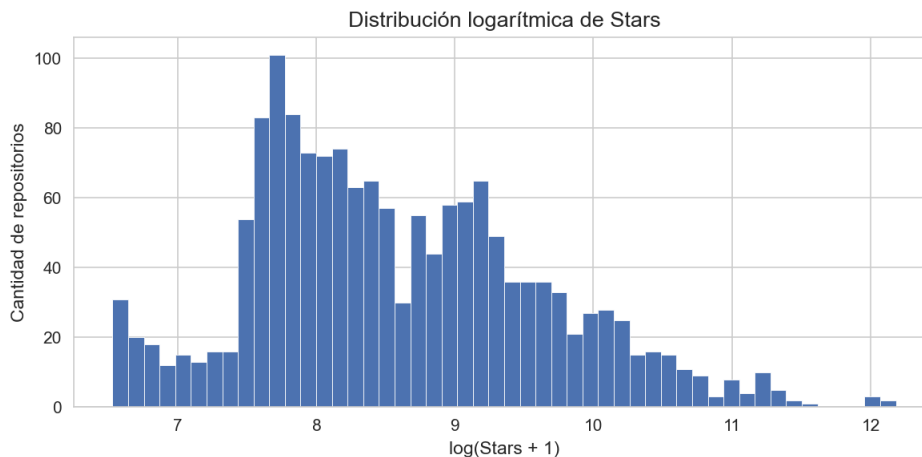


Figura 1. Distribución logarítmica de stars en el dataset.

Los boxplots de stars y forks confirman la presencia de una gran cantidad de valores atípicos, consistente con la estructura de "longtail" del ecosistema: la mayoría de repositorios tiene popularidad moderada, pero proyectos como scikit-learn o Keras acumulan decenas de miles de estrellas.

4.2 Lenguajes de Programación

Python domina ampliamente el dataset, lo que es coherente con su adopción como lenguaje estándar en Machine Learning, Deep Learning y análisis de datos. También aparecen Jupyter Notebook, JavaScript, TypeScript, C++ y Go, este último con presencia en proyectos de infraestructura y rendimiento computacional.

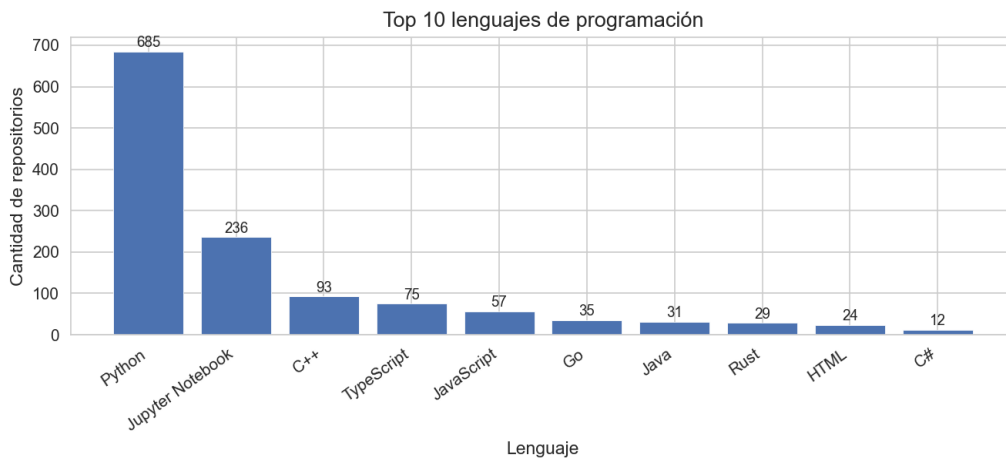


Figura 2. Top 10 lenguajes de programación en el dataset.

4.3 Correlaciones entre Variables

La matriz de correlación revela una relación positiva fuerte entre stars, forks y watchers. Esto indica que la participación comunitaria y la visibilidad pública están estrechamente ligadas. La variable stars_per_day también presenta correlaciones relevantes, sugiriendo que la velocidad de crecimiento es un buen predictor de popularidad.

En contraste, variables como size, description_length y topics_count muestran correlaciones débiles, lo que sugiere que el tamaño del código o la extensión de la documentación no determinan por sí solos el éxito de un proyecto.

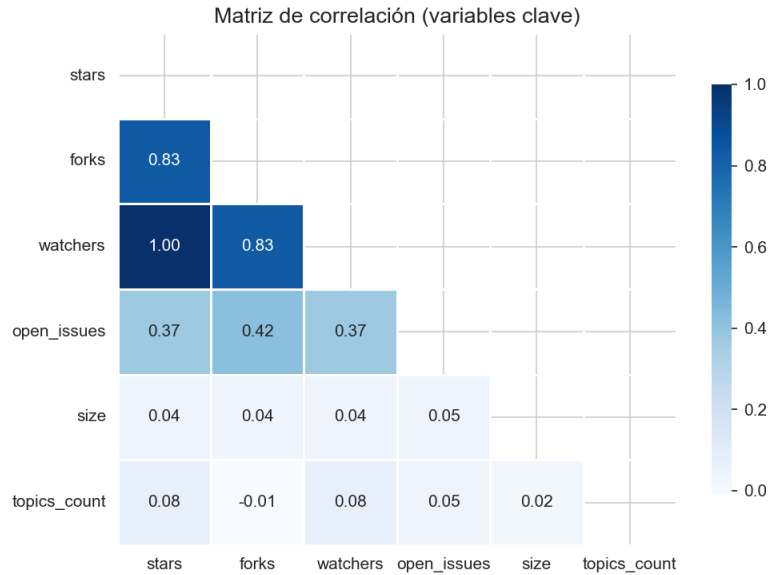


Figura 3. Matriz de correlación entre las variables numéricas principales.

4.4 Relación Stars vs. Forks

El diagrama de dispersión entre stars y forks evidencia una tendencia positiva clara. Los repositorios con mayor número de bifurcaciones tienden a acumular más estrellas, aunque existen casos donde un proyecto tiene muchos forks con relativamente pocas estrellas, lo que puede indicar proyectos de uso técnico más que de interés general.

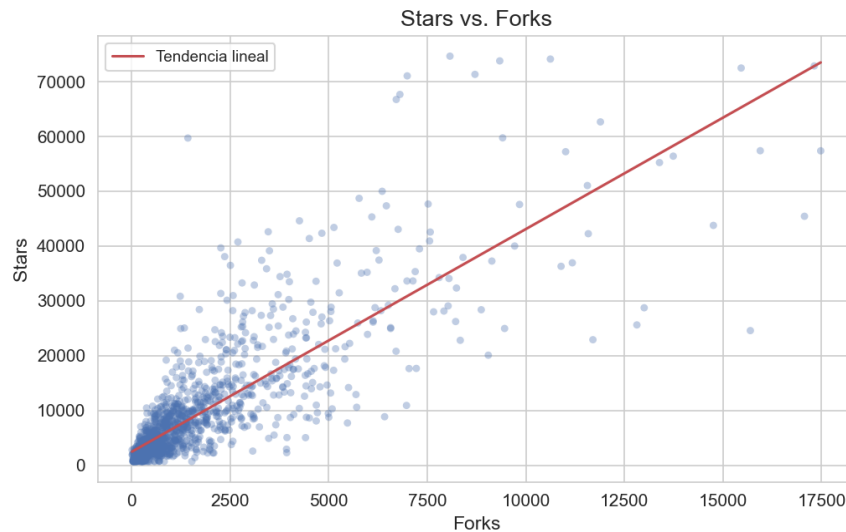


Figura 4. Relación entre stars y forks (sin valores atópicos extremos).

4.5 Evolución Temporal

Se observa un crecimiento acelerado en la creación de repositorios entre 2015 y 2020, coincidiendo con la expansión del Deep Learning y la popularización de TensorFlow y PyTorch. En años más recientes se nota una disminución, posiblemente porque los repositorios nuevos no han tenido tiempo suficiente para acumular estrellas.



Figura 5. Cantidad de repositorios creados por año.

5. Desarrollo del Modelo Predictivo

5.1 Preprocesamiento y Feature Engineering

Antes de entrenar los modelos se realizó el siguiente preprocesamiento:

- Eliminación de valores atípicos extremos: registros por encima del percentil 99 en stars (umbral: 75,090 estrellas).
- Creación de variables derivadas: forks_per_day, forks_per_size, log_forks, log_open_issues, entre otras.
- Codificación one-hot para la variable language, agrupando lenguajes con poca frecuencia en la categoría Other.
- Estandarización de variables numéricas mediante StandardScaler.

La matriz de features final quedó compuesta por 20 variables antes de la selección automática.

5.2 Modelos Evaluados

Se evaluaron seis algoritmos de regresión utilizando validación cruzada de 5 folds. Las métricas reportadas corresponden al promedio de los folds:

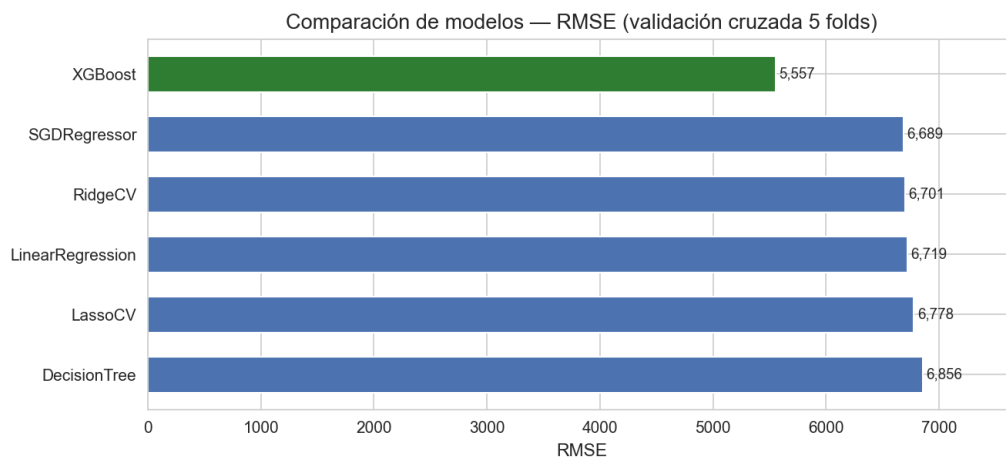


Figura 6. Comparación de modelos por RMSE en validación cruzada.

Modelo	RMSE	MAE	R2
XGBoost	5,557	2,983	0.730
SGDRegressor	6,689	3,814	0.612
RidgeCV	6,701	3,971	0.610
LinearRegression	6,719	3,807	0.608
LassoCV	6,778	3,868	0.600
DecisionTree	6,856	3,754	0.588

Tabla 2. Comparación de modelos por validación cruzada (5 folds).

XGBoost supero al resto de modelos por un margen considerable, obteniendo un RMSE de 5,557 y un R2 de 0.730, lo que indica que el modelo logra explicar el 73% de la varianza en la popularidad de los repositorios.

5.3 Selección de Variables con RFECV

Se aplico Recursive Feature Elimination with Cross-Validation (RFECV) sobre el modelo XGBoost para identificar el subconjunto optimo de variables. El proceso selecciono 17 de las 20 variables originales:

- forks (la variable mas importante segun feature importance: 25.4%)
- days_since_update, forks_per_day
- open_issues, size, topics_count, description_length, repo_age_days
- Variables dummies de lenguaje: C++, Go, HTML, Java, JavaScript, Jupyter Notebook, Other, Rust, TypeScript

El RMSE post-RFECV fue de 5,560, prácticamente idéntico al baseline, confirmando que la eliminación de las 3 variables no relevantes no perjudico el rendimiento.

5.4 Optimización de Hiperparámetros con Optuna

Se utilizo Optuna para realizar una búsqueda bayesiana de hiperparametros sobre XGBoost. Los mejores parámetros encontrados fueron:

- n_estimators: 298
- max_depth: 5
- learning_rate: 0.0106
- subsample: 0.941
- colsample_bytree: 0.823

El modelo final logro un RMSE de validación cruzada de 5,101 (con desviación estándar de 619), una mejora del 8% respecto al modelo baseline.

6. Resultados Obtenidos

Modelo	RMSE (CV)	MAE	R2
--------	-----------	-----	----

XGBoost baseline	5,557	2,983	0.730
XGBoost + RFECV	5,560	2,974	0.731
XGBoost + Optuna (CV)	5,101	N/A	N/A
XGBoost + Optuna (train)	3,153	1,889	0.916

Tabla 3. Resumen de métricas del modelo final.

Las variables más influyentes según el modelo XGBoost optimizado se muestran en la siguiente gráfica:

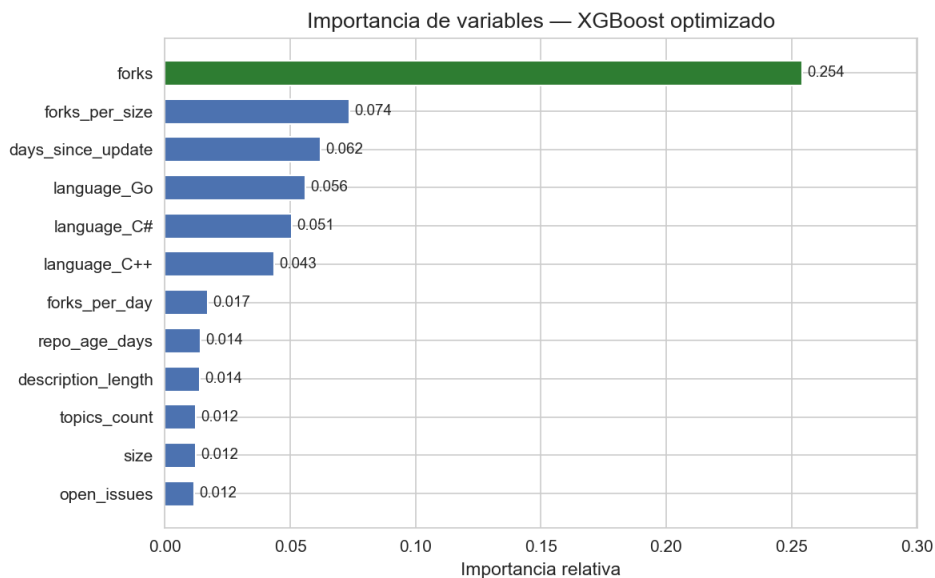


Figura 7. Importancia de variables en el modelo XGBoost optimizado.

La grafica de predicciones vs. valores reales muestra el comportamiento del modelo sobre el conjunto de datos:

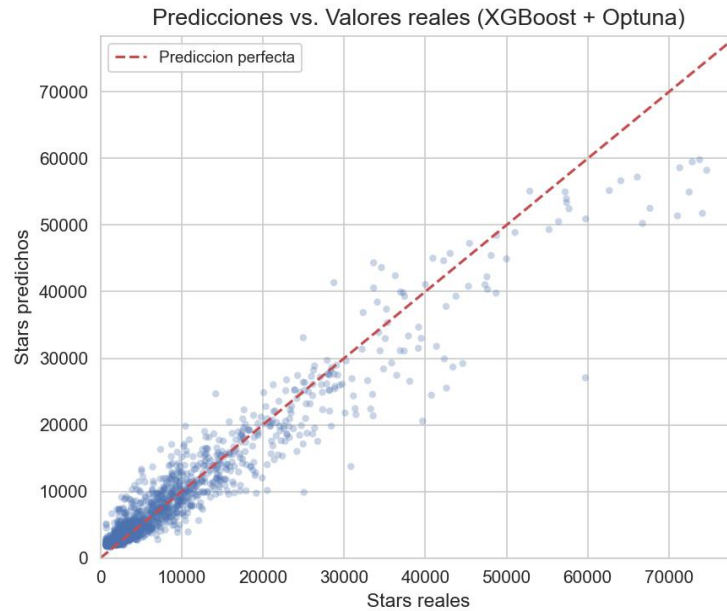


Figura 8. Predicciones vs. valores reales del modelo XGBoost + Optuna.

7. Análisis de Resultados

El modelo XGBoost optimizado con Optuna resulto ser claramente superior a los modelos lineales en todas las métricas evaluadas. Esto era esperable dado que la relación entre la popularidad de un repositorio y sus características no es lineal: un proyecto con muchos forks no acumula estrellas de forma proporcional, sino que existen umbrales y efectos de red que los árboles de decisión capturan mejor.

El hecho de que forks sea la variable más importante refuerza la idea de que la popularidad en GitHub tiene un componente de retroalimentación: los proyectos que son bifurcados son proyectos que la gente quiere usar y modificar, lo que a su vez atrae más visibilidad y estrellas. Esto no implica causalidad directa; más bien, ambos son síntomas de un proyecto que resuena con la comunidad.

La variable `days_since_update` también resulto relevante, lo que sugiere que los proyectos con mantenimiento reciente tienden a acumular más estrellas. Desde el punto de vista del usuario, un repositorio sin actualizaciones recientes genera desconfianza sobre su vigencia y compatibilidad.

En cuanto a los lenguajes, la presencia de Go y C# en el top de importancia puede explicarse porque los proyectos en estos lenguajes suelen estar orientados a infraestructura o herramientas de rendimiento, un nicho que atrae a comunidades especializadas con alta tasa de adopción.

El R^2 de 0.916 obtenido sobre el conjunto de entrenamiento sugiere un buen ajuste, aunque este valor debe interpretarse con cuidado ya que no se tiene un conjunto de prueba independiente separado desde el inicio. La validación cruzada con RMSE de 5,101 es la estimación más confiable del desempeño real del modelo.

8. Conclusiones

El proyecto permitió construir un modelo predictivo de popularidad de repositorios de Ciencia de Datos en GitHub con un desempeño razonable para la naturaleza del problema. Las principales conclusiones son:

- La popularidad en GitHub no es un fenómeno puramente estructural. El número de forks, el mantenimiento activo y la visibilidad comunitaria son mejores predictores que el tamaño del repositorio o la extensión de su documentación.
- XGBoost supero a todos los modelos lineales evaluados, confirmando que las relaciones entre variables y popularidad son no lineales y con posibles interacciones entre features.
- La optimización bayesiana con Optuna permitió mejorar el RMSE de validación cruzada en un 8% respecto al modelo baseline, con un costo computacional relativamente bajo.
- Python sigue siendo el lenguaje dominante en el ecosistema de Ciencia de Datos, aunque lenguajes como Go, C++ y Rust tienen presencia relevante en proyectos de infraestructura y rendimiento.
- El crecimiento más acelerado de repositorios en el área ocurrió entre 2015 y 2020, coincidiendo con la expansión del Deep Learning como tecnología de amplio uso.

Como trabajo futuro sería valioso incorporar variables de texto como el contenido del README mediante técnicas de NLP, implementar un pipeline de actualización automática del dataset a través de la API, y explorar modelos de series de tiempo para proyectar la trayectoria de popularidad de repositorios individuales.

9. Referencias

GitHub, Inc. (2024). GitHub REST API Documentation. <https://docs.github.com/en/rest>

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. <https://doi.org/10.1145/2939672.2939785>

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. Proceedings of the 25th ACM SIGKDD International Conference. <https://doi.org/10.1145/3292500.3330701>

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

10. Anexos

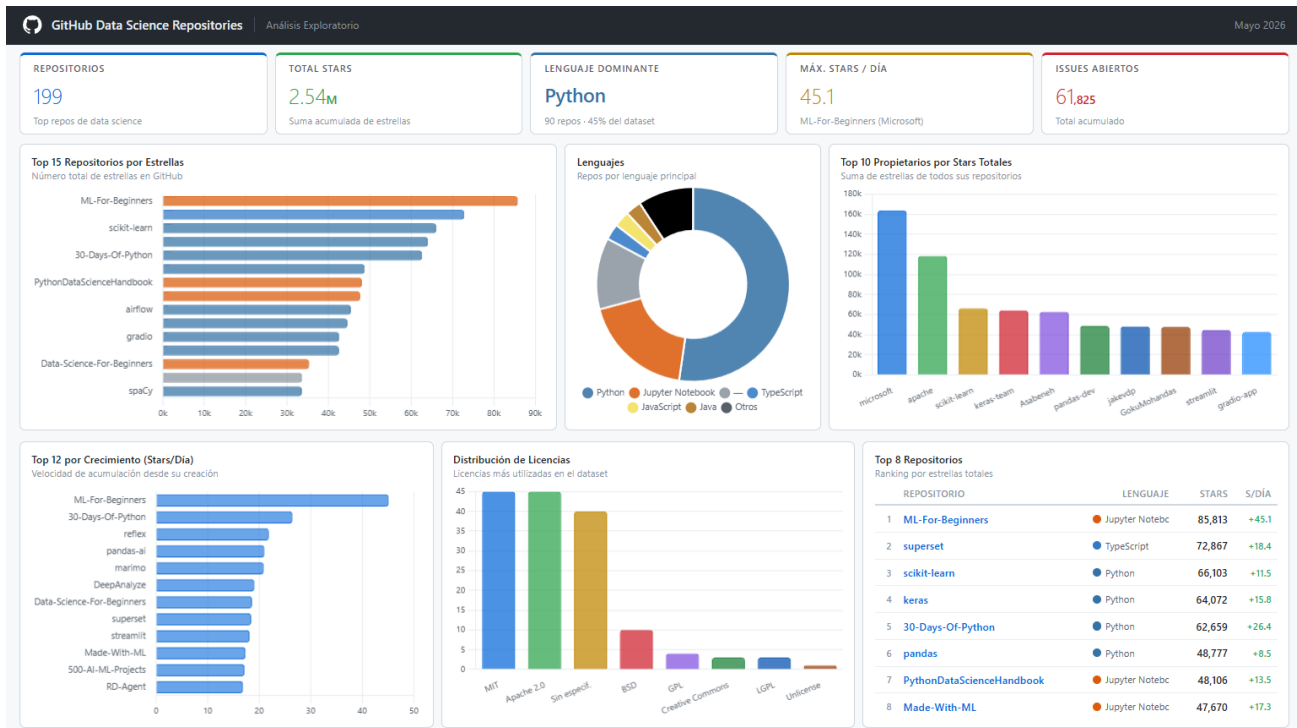


Figura 9. Dashboard – Análisis Exploratorio

10.1 Acceso a Github:

https://github.com/CarFangM/proyecto_final_ml-.git